# VenueOS

## AI-Powered Crowd Lifecycle Management Platform

*Product Specification & Build Document*

| | |
|---|---|
| **Hackathon** | PromptWars Virtual 2026 — Google for Developers x Hack2Skill |
| **Submission deadline** | April 19, 2026 — 11:59 PM IST |
| **Mandatory tools** | Google Antigravity (IDE), Gemini API, Cloud Run, GitHub |
| **Document purpose** | Full build specification for Antigravity-assisted vibe coding |
| **Version** | 1.0 — Final |

| | |
|---|---|
| **NOTE** | This document is the single source of truth for building VenueOS. Every functional requirement, data model, API call, UI screen, and deployment instruction is specified here. Antigravity should be able to build the entire solution from this document without additional context. |

# 1. Product Overview

What VenueOS is, who it serves, and why it exists

## 1.1 Product Summary

VenueOS is a full-stack, AI-powered crowd lifecycle management platform for large-scale sporting events. It addresses the entire timeline of an attendee's event day — from pre-event planning by organisers, through real-time crowd monitoring, to controlled exit after the match.

The platform operates across two distinct user interfaces: an Organiser Dashboard for venue staff, and an Attendee Mobile Web App for ticket holders. Both interfaces share a common Firestore database and are coordinated by Gemini AI, which acts as the intelligent planning and operations engine.

Primary context: IPL and international cricket matches at large Indian venues such as MA Chidambaram (Chennai), Wankhede (Mumbai), and Eden Gardens (Kolkata), each hosting 40,000 to 80,000 attendees.

## 1.2 Core Problem Being Solved

| Problem | Impact |
|---|---|
| **Crowd surges at gates** | All attendees attempt to enter simultaneously, causing dangerous bottlenecks |
| **No pre-event spatial planning** | Road blocking, gate assignments, and staff positions are decided ad hoc on the day |
| **Zero real-time intelligence** | Organisers cannot see crowd density across the venue or predict where problems will develop |
| **Stampede risk at exits** | Post-match mass exodus with no slot management or directional guidance |
| **Attendee disorientation** | No live map, no amenity finder, no way to report issues in real time |

## 1.3 Two User Types

| User | Role & Interface |
|---|---|
| **Organiser / Venue Staff** | Uses the desktop Organiser Dashboard to plan crowd management pre-event and monitor operations in real time on event day |

| | |
|---|---|
| **Attendee** | Uses the mobile web app after authenticating via ticket QR scan + OTP. Performs slot booking, receives QR pass, navigates to venue, reports incidents |

# 2. Functional Requirements

Every feature the system must deliver, grouped by module

## Module A — GIS Road & Gate Planner (Organiser)

Purpose: Allow organisers to define the venue map, mark entry/exit gates, map roads to gates, and use Gemini to generate an optimal road-blocking and crowd routing plan before the event.

**FR-A1 Venue map setup** *[Must Have]*

Organiser can create a venue profile by entering venue name, city, and capacity. The system embeds a Google Map centred on the venue. Organiser can place polygon markers for the venue boundary, gate markers (labelled Gate 1, Gate 2... Gate N), road entry markers on surrounding roads, and amenity markers (water, restroom, first aid, food stall).

**FR-A2 Gate-to-road assignment** *[Must Have]*

For each gate, the organiser can assign one or more approach roads from the road markers placed on the map. The UI shows a list of all gates with a multi-select dropdown of roads. Each road-gate pair is saved to Firestore.

**FR-A3 Perimeter definition** *[Must Have]*

Organiser draws a circular or polygonal geo-fence perimeter around the venue on the map. The perimeter radius is saved. This perimeter is used for attendee slot validation on event day.

**FR-A4 Gemini road-blocking plan generation** *[Must Have]*

Organiser inputs: expected crowd size, event start time, and number of gates. Pressing 'Generate Plan' calls the Gemini API with venue data, gate-road assignments, and crowd size. Gemini returns a structured plan: which roads to block, which roads to keep open, recommended traffic diversion instructions per gate, and staff deployment positions. The plan is rendered as a text report and as colour-coded overlays on the map.

**FR-A5 Plan export** *[Must Have]*

The generated plan can be exported as a PDF or copied as plain text for distribution to ground teams.

## Module B — Batch & Slot Scheduler (Organiser)

Purpose: Allow organisers to divide all attendees into batches of 100-200 people, assign each batch a gathering point and time slot for entry and exit, and use Gemini to compute this automatically.

**FR-B1 Batch generation via Gemini** *[Must Have]*

Organiser inputs: total attendee count, number of gates, event duration, number of outer gathering zones available, and transport capacity per trip. Pressing 'Generate Batches' calls Gemini which computes: number of batches, batch size per gate, entry time window per batch (spaced to prevent simultaneous gate rushes), exit time window per batch. Gemini outputs a structured JSON schedule.

**FR-B2 Batch code assignment** *[Must Have]*

Each batch is assigned a unique code (e.g. B-A1, B-A2, B-B1...). The first letter indicates the gate (A = Gate 1, B = Gate 2), the number indicates the sequence. Codes are stored in Firestore under the venue event document.

**FR-B3 Gathering point assignment** *[Must Have]*

Each batch is assigned to one outer gathering zone. Gathering zones are locations placed on the map by the organiser (FR-A1 extension). Each zone can hold a maximum of N attendees (configurable). The system distributes batches across zones evenly.

**FR-B4 Entry and exit slot display** *[Must Have]*

The organiser dashboard shows a full schedule table: Batch Code | Gate | Gathering Zone | Entry Window | Exit Window | Transport Departure Time. The schedule is editable — organisers can manually adjust time windows and save.

**FR-B5 Attendee slot notification** *[Must Have]*

When the schedule is finalised, the system marks it as 'Published'. Attendees who have authenticated and booked their slot receive their batch code, gathering zone address, entry window, and exit window in their app.

## Module C — Outer Gathering Zone & Transport System (Organiser + Attendee)

Purpose: Manage the outer gathering zones where attendees assemble by batch before being transported to the venue. The gathering zone is deliberately placed at an open, spacious location away from the venue perimeter to avoid pre-event crowding near the stadium.

**FR-C1 Gathering zone map setup** *[Must Have]*

Organiser places gathering zone markers on the map at locations that are: (a) a minimum of 500m from the venue perimeter, (b) have sufficient open space for the batch size. Each zone has a name, GPS coordinates, and maximum capacity. Zones are visible to attendees on their live map.

**FR-C2 Transport fleet setup** *[Must Have]*

Organiser registers available transport units (buggies, buses, or shuttles) with: transport ID, type, capacity, and assigned route (gathering zone to venue gate). This enables the dispatch dashboard.

**FR-C3 Batch transport dispatch** *[Must Have]*

When a batch's entry window is approaching (configurable threshold, default 20 minutes), the organiser dashboard shows a 'Dispatch' button for that batch. Clicking it: marks the batch as 'In Transit' in Firestore, notifies attendees in that batch that their transport is ready, and records dispatch timestamp.

**FR-C4 QR scan at transport boarding** *[Must Have]*

Each transport boarding point has a staff-operated QR scanner (browser-based, using the device camera). When an attendee scans their QR pass, the system validates: (a) batch code matches the dispatched batch, (b) attendee is not already marked as boarded. On success, the system marks the attendee as 'Boarded' in Firestore. Invalid scans show an error with the reason.

**FR-C5 Attendee navigation to gathering zone** *[Must Have]*

The attendee app shows turn-by-turn directions from the attendee's current GPS location to their assigned gathering zone using Google Maps Directions API (walking or driving mode selectable). Navigation activates automatically when the attendee opens the app after slot booking.

## Module D — Geo-fenced Perimeter Control (Organiser + Attendee)

Purpose: Enforce slot-based entry by detecting when an attendee enters the venue perimeter and validating their entry slot before allowing them to proceed to the gate.

**FR-D1 Perimeter detection on attendee app** *[Must Have]*

When the attendee app is open and the attendee's GPS location enters the perimeter drawn by the organiser (FR-A3), the app checks: (a) Is the attendee's batch entry window currently active? If yes: show green status 'You are cleared to enter. Proceed to [Gate X].' If no: show amber warning 'You have arrived early. Your entry window is [time]. Please wait at [Gathering Zone name].' and display navigation back to the gathering zone.

**FR-D2 QR scan at perimeter gate** *[Must Have]*

A staff-operated QR scanner at the perimeter boundary validates: (a) batch entry window is active, (b) attendee is marked as 'Boarded' (came via transport), (c) QR pass is not expired or revoked. On success, marks attendee as 'Perimeter Cleared' in Firestore.

**FR-D3 Early arrival redirect** *[Must Have]*

If an attendee's GPS enters the perimeter before their entry window, the app automatically shows a push notification and in-app alert with navigation directions back to their assigned gathering zone and their correct entry time.

## Module E — QR Pass System (Attendee + All Checkpoints)

Purpose: A single QR code governs all attendee checkpoints. The QR pass is generated only after slot booking is confirmed and contains all information needed at every stage.

**FR-E1 QR pass generation** *[Must Have]*

After an attendee completes slot booking (selects entry window and exit window), the system calls a Cloud Function that generates a QR pass. The QR encodes a signed JSON payload containing: attendee ID, event ID, batch code, gate assignment, gathering zone ID, entry window start/end timestamps, exit window start/end timestamps, and a HMAC signature for tamper detection. The QR is displayed in the attendee app and can be saved to the device camera roll.

**FR-E2 QR scanner (staff)** *[Must Have]*

Any staff member can open the scanner view on their phone browser. The scanner uses the device camera via the jsQR library. On scan, it decodes the payload, verifies the HMAC signature, and checks the appropriate checkpoint rule (transport boarding, perimeter, gate entry, exit). Results are shown instantly with a green/red colour indicator and written to Firestore.

**FR-E3 Checkpoint stages** *[Must Have]*

The QR pass tracks six checkpoint stages in order: (1) Slot Booked, (2) Transport Boarded, (3) Perimeter Cleared, (4) Gate Entered, (5) Inside Venue, (6) Exited. Each stage is timestamped in Firestore. Attempting to scan a stage out of order shows an error.

**FR-E4 Exit slot enforcement** *[Must Have]*

On exit, the scanner checks that the attendee's exit window is currently active. If the attendee tries to exit before their window, the scanner shows an amber warning with their exit window time. In an emergency, organisers can override from the dashboard.

## Module F — Live Crowd Intelligence Dashboard (Organiser)

Purpose: Real-time crowd density visualisation and Gemini-powered anomaly detection for organisers and staff.

**FR-F1 Crowd density heatmap** *[Must Have]*

The organiser dashboard shows a Google Maps heatmap layer over the venue and surroundings. Density data is sourced from: (a) GPS pings from attendee app (sent every 30 seconds while app is open), (b) QR scan events at each checkpoint (each scan adds a density point at the scan location). The heatmap updates every 60 seconds. Intensity zones: Low (green), Medium (amber), High (red), Critical (dark red).

**FR-F2 Per-gate crowd counter** *[Must Have]*

The dashboard shows a live count card for each gate: people currently in queue (estimated from GPS pings within 50m of gate), people entered in last 10 minutes (from gate scan events), percentage of gate capacity used.

**FR-F3 Gemini anomaly detection** *[Must Have]*

Every 60 seconds, a Cloud Run scheduled job assembles a density snapshot (headcount per zone, rate of change per zone, match state, time to next natural break) and calls Gemini API. Gemini

analyses the snapshot and returns: predicted congestion zones in the next 5-15 minutes, confidence level, recommended action for each zone (redirect attendees, deploy more staff, open additional gate), and natural language summary. The dashboard shows these predictions as highlighted zones with tooltips.

**FR-F4 Auto-drafted staff instructions** *[Must Have]*

When Gemini predicts a critical congestion event (confidence > 70%), the system auto-generates a staff instruction card on the dashboard: 'Send 3 additional staff to Gate 3. Announce redirect to Gate 7.' Organisers can one-click approve or edit before sending.

**FR-F5 Attendee redirection push** *[Must Have]*

When an organiser approves a redirect instruction, the system sends an in-app notification to all attendees currently within 200m of the congested gate, directing them to the alternative gate.

## Module G — Attendee Mobile Web App

Purpose: The attendee-facing interface. Accessible only after ticket scan + OTP authentication. Not publicly accessible.

**FR-G1 Ticket scan and OTP authentication** *[Must Have]*

The attendee visits the VenueOS web app URL (or scans a QR on their physical ticket). The app prompts them to enter their registered mobile number. Firebase Auth sends an OTP SMS. On verification, the attendee is authenticated and their profile is loaded from Firestore (matched by phone number to the ticketing database).

**FR-G2 Slot booking screen** *[Must Have]*

After authentication, the attendee selects an entry time slot from available windows (slots not yet at capacity). They also select an exit time slot. On confirmation, the system assigns them a batch code and gathering zone, generates their QR pass (FR-E1), and saves all data to Firestore.

**FR-G3 Home screen / dashboard** *[Must Have]*

Post-booking home screen shows: QR pass (large, tappable to expand), batch code and gathering zone name, entry window time, exit window time, current checkpoint stage with progress indicator, and a live countdown to their entry window if not yet active.

**FR-G4 Navigation screen** *[Must Have]*

Tap 'Navigate' to get Google Maps turn-by-turn directions to the gathering zone (pre-entry) or to the assigned gate (once inside perimeter). Uses browser Geolocation API for current position and Google Maps Directions API for routing.

**FR-G5 Live venue map** *[Must Have]*

Shows a full-screen Google Map of the venue with: all gate markers, amenity markers (water, restroom, first aid, food stall), gathering zones, the perimeter boundary, and a simplified crowd density overlay (colour-coded zones). Attendee's own location shown as a blue dot.

**FR-G6 Amenity finder** *[Must Have]*

A bottom sheet UI lists all amenities by category (water, restrooms, food, first aid). Tapping an amenity centres the map on it and shows walking distance. A 'Report Issue' button opens the incident reporting form pre-filled with the amenity location.

**FR-G7 Incident and amenity reporting** *[Must Have]*

The report form has two types: (a) General incident: description text field + optional photo/video upload (max 30 seconds video, Cloud Storage). (b) Amenity issue: amenity pre-selected from map, issue type dropdown (broken, out of stock, unclean, other), optional photo. On submit, Gemini API categorises the report, assigns a priority level, and writes to Firestore with routing to the relevant staff role. Attendee sees a confirmation with a report reference number.

**FR-G8 Notifications** *[Must Have]*

The app shows in-app banners for: entry window approaching (30 minutes before), transport dispatched (immediate), crowd redirect advisory (immediate), exit window approaching (30 minutes before). If browser push notifications are enabled, these also appear as system notifications.

## Module H — Authentication & Access Control

**FR-H1 Organiser authentication** *[Must Have]*

Organisers log in with Google Sign-In (Firebase Auth with Google provider). Only pre-approved Google accounts (listed in a Firestore 'admins' collection) can access the organiser dashboard. Unauthorised accounts see an access denied screen.

**FR-H2 Attendee authentication** *[Must Have]*

Phone OTP via Firebase Auth. The attendee app is not publicly accessible — all routes redirect to the auth screen if not authenticated. Session persists across page reloads using Firebase's session persistence.

**FR-H3 Staff scanner access** *[Must Have]*

Staff access the QR scanner via a direct URL with a staff token (short-lived JWT generated by organisers from the dashboard). No full account needed — the token grants scanner-only access for a defined event day.

# 3. Technical Requirements

Architecture, stack, data models, API contracts, and deployment

## 3.1 Technology Stack

| Component | Purpose | Status |
|---|---|---|
| **Google Antigravity** | IDE — entire codebase built via prompt-driven agentic coding | Mandatory |
| **Gemini API (gemini-1.5-pro)** | Planning, anomaly detection, batch scheduling, incident triage, redirection | Mandatory |
| **Google Cloud Run** | Containerised backend — Python (FastAPI) or Node.js (Express). Auto-scales for event day spikes | Mandatory |
| **Cloud Firestore** | Real-time NoSQL database. All attendee data, batch codes, scan events, reports | Mandatory |
| **Firebase Auth** | Phone OTP for attendees, Google Sign-In for organisers | Mandatory |
| **Google Maps Platform** | Maps embed, heatmap layer, directions API, geocoding API, geolocation | Mandatory |
| **Cloud Storage** | Incident photo and video uploads from attendees | Mandatory |
| **Cloud Functions** | QR pass generation, scheduled Gemini density analysis (every 60s), report routing | Mandatory |
| **Cloud Scheduler** | Triggers the density analysis Cloud Function every 60 seconds | Mandatory |
| **GitHub** | Source repository. CI/CD pipeline to Cloud Run via Cloud Build | Mandatory |
| **jsQR (browser library)** | Browser-based QR code scanning on staff devices (no native app needed) | Frontend |
| **qrcode.js (browser library)** | QR code rendering in the attendee app | Frontend |

## 3.2 Repository Structure

The GitHub repository should follow this structure:

| Path | Contents |
|---|---|
| `/backend/` | FastAPI or Express backend — API routes, Gemini service, Firestore service |

| | |
|---|---|
| `/backend/Dockerfile` | Container config for Cloud Run deployment |
| `/backend/main.py (or index.js)` | Entry point — all API route definitions |
| `/backend/services/gemini.py` | All Gemini API calls encapsulated here |
| `/backend/services/firestore.py` | All Firestore read/write operations |
| `/backend/services/qr.py` | QR pass generation and HMAC signing |
| `/frontend-organiser/` | Organiser dashboard — HTML/CSS/JS or React |
| `/frontend-attendee/` | Attendee mobile web app — HTML/CSS/JS or React |
| `/functions/` | Cloud Functions — density analysis, QR generation trigger |
| `/firestore.rules` | Firestore security rules |
| `/cloudbuild.yaml` | CI/CD config — builds and deploys to Cloud Run on push to main |
| `/README.md` | Setup instructions, env variables, deployment steps |

## 3.3 Firestore Data Model

All collections and their document schemas. Use these exact field names throughout the codebase.

### Collection: events

One document per event. Document ID = event slug (e.g. 'ipl-csk-mi-2026-04-19').

| Field | Type & Description |
|---|---|
| **eventId** | string — unique slug |
| **venueName** | string — e.g. 'MA Chidambaram Stadium' |
| **venueCity** | string |
| **eventDate** | timestamp |
| **eventStartTime** | timestamp |
| **totalCapacity** | number |
| **gates** | array of GateObject { gateId, gateName, location: {lat, lng}, capacity, assignedRoads: [string] } |
| **gatheringZones** | array of ZoneObject { zoneId, zoneName, location: {lat, lng}, maxCapacity, assignedGate } |
| **perimeterCenter** | GeoPoint |
| **perimeterRadiusMeters** | number |
| **amenities** | array of AmenityObject { amenityId, type, name, location: {lat, lng}, status: 'open'\|'issue'\|'closed' } |

| | |
|---|---|
| **roadPlan** | string — Gemini-generated road blocking plan text |
| **batchSchedule** | array of BatchObject (see batches collection) |
| **schedulePublished** | boolean |
| **createdAt** | timestamp |

## Collection: attendees

One document per attendee. Document ID = Firebase Auth UID.

| Field | Type & Description |
|---|---|
| **uid** | string — Firebase Auth UID |
| **phone** | string — E.164 format |
| **name** | string |
| **ticketId** | string — from ticketing system |
| **eventId** | string — ref to events collection |
| **seatSection** | string — e.g. 'North Stand B' |
| **batchCode** | string — assigned after slot booking, e.g. 'B-A3' |
| **gateAssignment** | string — gateId |
| **gatheringZoneId** | string — zoneId |
| **entryWindowStart** | timestamp |
| **entryWindowEnd** | timestamp |
| **exitWindowStart** | timestamp |
| **exitWindowEnd** | timestamp |
| **qrPayload** | string — signed QR JSON payload |
| **checkpointStage** | string — enum: 'booked' \| 'boarded' \| 'perimeter_cleared' \| 'gate_entered' \| 'inside' \| 'exited' |
| **checkpointTimestamps** | map { boarded_at, perimeter_cleared_at, gate_entered_at, exited_at } |
| **createdAt** | timestamp |

## Collection: scan_events

One document per QR scan. Used for density calculations and audit trail.

| Field | Type & Description |
|---|---|
| **scanId** | string — auto-generated |
| **attendeeId** | string — ref to attendees |

| | |
|---|---|
| **eventId** | string |
| **checkpointType** | string — enum: 'transport' \| 'perimeter' \| 'gate' \| 'exit' |
| **checkpointId** | string — gateId or zoneId |
| **location** | GeoPoint — location of the scanner |
| **result** | string — 'success' \| 'error' |
| **errorReason** | string — if result is error |
| **scannedAt** | timestamp |
| **scannedBy** | string — staff token or staff UID |

## Collection: gps_pings

GPS location pings from attendee app. One document per ping. TTL: auto-deleted after 4 hours via Firestore TTL policy.

| Field | Type & Description |
|---|---|
| **pingId** | string — auto-generated |
| **attendeeId** | string |
| **eventId** | string |
| **location** | GeoPoint |
| **timestamp** | timestamp |
| **expireAt** | timestamp — timestamp + 4 hours (TTL field) |

## Collection: density_snapshots

Output of the 60-second density analysis job. Organiser dashboard reads the latest snapshot.

| Field | Type & Description |
|---|---|
| **snapshotId** | string — auto-generated |
| **eventId** | string |
| **timestamp** | timestamp |
| **zoneData** | array of ZoneDensity { zoneId, headcount, rateOfChange, intensityLevel: 'low'\|'medium'\|'high'\|'critical' } |
| **geminiPrediction** | string — Gemini natural language summary |
| **predictedCongestionZones** | array of { zoneId, predictedPeakMinutes, confidence, recommendedAction } |
| **alertsGenerated** | array of AlertObject { alertId, zoneId, message, targetAttendeeRadius, status: 'pending'\|'approved'\|'sent' } |

## Collection: incident_reports

| Field | Type & Description |
|---|---|
| **reportId** | string — auto-generated |
| **attendeeId** | string |
| **eventId** | string |
| **reportType** | string — 'general_incident' \| 'amenity_issue' |
| **amenityId** | string — if amenity_issue, ref to amenity |
| **description** | string |
| **issueType** | string — for amenity: 'broken' \| 'out_of_stock' \| 'unclean' \| 'other' |
| **mediaUrls** | array of strings — Cloud Storage URLs |
| **location** | GeoPoint — attendee location at time of report |
| **geminiCategory** | string — Gemini-assigned category |
| **geminiPriority** | string — 'low' \| 'medium' \| 'high' \| 'urgent' |
| **routedToRole** | string — staff role to handle: 'maintenance' \| 'security' \| 'medical' \| 'operations' |
| **status** | string — 'open' \| 'acknowledged' \| 'resolved' |
| **createdAt** | timestamp |

## 3.4 Backend API Endpoints

All endpoints are served by the Cloud Run backend. Base URL: https://[cloud-run-service-url]/api/v1

### Auth & Attendee

| Endpoint | Description | Access |
|---|---|---|
| **POST /auth/verify-otp** | Verify Firebase OTP token, return attendee profile or prompt registration | Public |
| **GET /attendee/:uid** | Get full attendee profile including batch assignment and QR payload | Auth required |
| **POST /attendee/:uid/book-slot** | Book entry/exit slot. Body: { entryWindowId, exitWindowId }. Returns batch assignment + QR payload | Auth required |
| **GET /attendee/:uid/qr** | Get the current QR pass payload as a signed string | Auth required |

### Organiser — Event & Planning

| Endpoint | Description | Access |
|---|---|---|
| **POST /organiser/events** | Create a new event. Body: all fields from events schema | Organiser |
| **GET /organiser/events/:eventId** | Get full event data | Organiser |
| **PUT /organiser/events/:eventId/gates** | Update gate and road assignments | Organiser |
| **PUT /organiser/events/:eventId/perimeter** | Update perimeter GeoPoint and radius | Organiser |
| **PUT /organiser/events/:eventId/amenities** | Update amenities list | Organiser |
| **POST /organiser/events/:eventId/generate-road-plan** | Call Gemini to generate road-blocking plan. Returns plan text + structured JSON | Organiser |
| **POST /organiser/events/:eventId/generate-batches** | Call Gemini to generate batch schedule. Body: { totalAttendees, transportCapacity }. Returns full batchSchedule array | Organiser |
| **PUT /organiser/events/:eventId/publish-schedule** | Mark schedule as published, triggers attendee notifications | Organiser |

### Operations — Real-time

| Endpoint | Description | Access |
|---|---|---|
| **GET /organiser/events/:eventId/density** | Get latest density snapshot from Firestore | Organiser |
| **POST /organiser/events/:eventId/dispatch-batch** | Dispatch a batch for transport. Body: { batchCode }. Updates Firestore, notifies attendees | Organiser |
| **POST /organiser/events/:eventId/alerts/:alertId/approve** | Approve a Gemini-generated alert. Triggers attendee push notification | Organiser |
| **POST /scanner/validate** | Validate a QR scan. Body: { qrPayload, checkpointType, checkpointId, scannerLocation }. Returns { valid, stage, attendeeName, errorReason } | Staff token |

### Attendee — Live Map & Reporting

| Endpoint | Description | Access |
|---|---|---|
| **POST /attendee/:uid/gps-ping** | Record a GPS ping. Body: { location: {lat, lng} }. Writes to gps_pings collection | Auth required |
| **GET /events/:eventId/map-data** | Get all gates, amenities, gathering zones, and simplified density zones for the attendee map | Auth required |
| **POST /attendee/:uid/report** | Submit an incident or amenity report. Body: full report fields. Triggers Gemini categorisation | Auth required |
| **GET /attendee/:uid/notifications** | Get list of unread notifications for the attendee | Auth required |

## 3.5 Gemini API Prompts

These are the exact prompt templates to use when calling the Gemini API. Use gemini-1.5-pro model for all calls. All responses should be requested as JSON unless otherwise noted.

### Prompt 1 — Road & Gate Planning

| | |
|---|---|
| **SYSTEM** | You are an expert crowd management consultant for large sporting venues in India. You think in terms of safety, flow efficiency, and stampede prevention. Always output valid JSON. |

| | |
|---|---|
| **USER TEMPLATE** | You are planning crowd management for {eventName} at {venueName}, {venueCity}. Expected attendance: {totalAttendees}. Event start: {eventStartTime}. Gates available: {gateList}. Entry roads and their gate assignments: |

| | |
|---|---|
| | {roadGateMapping}. Generate a detailed road-blocking and crowd routing plan. Return JSON with: { "blockedRoads": [{"roadName": string, "reason": string}], "openRoads": [{"roadName": string, "designatedGate": string, "instructions": string}], "staffPositions": [{"location": string, "role": string, "count": number}], "summary": string } |

## Prompt 2 — Batch & Slot Generation

| | |
|---|---|
| **SYSTEM** | You are an expert event operations planner. Generate batch schedules that prevent simultaneous crowd surges at gates. Always output valid JSON. Batch sizes should be between 100 and 200 people. |

| | |
|---|---|
| **USER TEMPLATE** | Plan batch entry and exit scheduling for {eventName}. Total attendees: {totalAttendees}. Number of gates: {numGates}. Gate capacities: {gateCapacities}. Event start: {eventStartTime}. Event end (estimated): {eventEndTime}. Transport capacity per trip: {transportCapacity}. Number of gathering zones: {numZones}. Generate a complete batch schedule. Return JSON with: { "batches": [{ "batchCode": string, "gateId": string, "gatheringZoneId": string, "size": number, "entryWindowStart": "HH:MM", "entryWindowEnd": "HH:MM", "exitWindowStart": "HH:MM", "exitWindowEnd": "HH:MM", "transportDepartureTime": "HH:MM" }], "summary": string } |

## Prompt 3 — Density Anomaly Detection (runs every 60s)

| | |
|---|---|
| **SYSTEM** | You are an AI operations co-pilot for a live sporting event. Analyse crowd density data and predict congestion before it happens. Be specific about time estimates. Always output valid JSON. |

| | |
|---|---|
| **USER TEMPLATE** | Live crowd data for {eventName} at {currentTime}. Match state: {matchState} (e.g. '14th over of 1st innings', 'drinks break in 8 minutes', 'last over'). Zone density: {zoneDataJson}. Each zone has: headcount in last 5 minutes, rate of change per minute (positive = growing), gate capacity. Predict congestion for the next 15 minutes. Return JSON with: { "predictions": [{ "zoneId": string, "predictedPeakMinutes": number, "confidence": number (0-1), "severity": "low"\|"medium"\|"high"\|"critical", "recommendedAction": string }], "redirectAdvisories": [{ "fromZoneId": string, "toZoneId": string, "messageToAttendees": string, "targetRadiusMeters": number }], "staffInstructions": [{ "instruction": string, "zone": string, "urgency": "low"\|"medium"\|"high" }], "summary": string } |

## Prompt 4 — Incident Report Categorisation

| | |
|---|---|
| **SYSTEM** | You are an operations triage assistant at a sporting venue. Classify incoming incident reports and route them to the right team. Always output valid JSON. |

| | |
|---|---|
| **USER TEMPLATE** | Incident report at {eventName}. Time: {timestamp}. Reporter location: {location}. Report type: {reportType}. Description: {description}. Amenity affected (if any): {amenityName}. Issue type (if any): {issueType}. Classify this report. Return JSON with: { "category": "medical"\|"security"\|"maintenance"\|"operations"\|"fire"\|"crowd_safety", "priority": "low"\|"medium"\|"high"\|"urgent", "routedToRole": "medical_team"\|"security"\|"maintenance"\|"operations_manager", "actionRequired": string, "estimatedResponseTime": string } |

## 3.6 QR Pass — Payload Schema & Signing

| | |
|---|---|
| **PAYLOAD SCHEMA** | { "v": 1, "aid": "attendee_uid", "eid": "event_id", "bc": "batch_code", "gid": "gate_id", "zid": "zone_id", "ews": 1713456000, "ewe": 1713459600, "xws": 1713492000, "xwe": 1713495600, "sig": "hmac_sha256_hex" } |

Field key: v=version, aid=attendeeId, eid=eventId, bc=batchCode, gid=gateId, zid=zoneId, ews=entryWindowStart (Unix timestamp), ewe=entryWindowEnd, xws=exitWindowStart, xwe=exitWindowEnd, sig=HMAC-SHA256 signature.

| | |
|---|---|
| **SIGNING** | Signing key = HMAC_SECRET env variable (set in Cloud Run). Sign the JSON string of all fields except 'sig' using HMAC-SHA256. The scanner validates by re-computing the HMAC and comparing to the sig field. Any mismatch = invalid/tampered QR. |

| | |
|---|---|
| **QR FORMAT** | Encode the entire JSON payload (including sig) as a base64 string, then encode that as a QR code using qrcode.js. Error correction level: H (highest). QR code size: 300x300px minimum for reliable scanning. |

## 3.7 Perimeter Detection Logic

Implement this logic in the attendee app JavaScript:

| | |
|---|---|
| **ALGORITHM** | 1. Every 30 seconds, call navigator.geolocation.getCurrentPosition(). 2. Compute distance from current position to perimeterCenter using the Haversine formula. 3. If distance <= perimeterRadiusMeters: fetch attendee's entryWindowStart and entryWindowEnd from Firestore. 4. If current time is within entry window: show green 'Cleared to enter' UI. 5. If current time is before entry window: show amber warning with remaining time and navigation back to gathering zone. 6. If attendee is outside perimeter: normal UI, show navigation to gathering zone. |

| | |
|---|---|
| **HAVERSINE** | function haversineDistanceMeters(lat1, lng1, lat2, lng2) { const R = 6371000; const dLat = (lat2-lat1)*Math.PI/180; const dLng = (lng2-lng1)*Math.PI/180; const a = Math.sin(dLat/2)**2 + Math.cos(lat1*Math.PI/180)*Math.cos(lat2*Math.PI/180)*Math.sin(dLng/2)**2; return R * 2 * Math.atan2(Math.sqrt(a), Math.sqrt(1-a)); } |

# 4. Process Flows

Step-by-step flows for every critical path in the system

## 4.1 Pre-Event Organiser Planning Flow

This flow is completed by the organiser in the days before the event.

1. Organiser logs in with Google account (Firebase Auth). System checks Firestore 'admins' collection — access denied if not listed.
2. Organiser creates a new event: enters venue name, city, date, start time, and total capacity.
3. Venue map screen loads with Google Maps centred on venue. Organiser places: venue boundary polygon, gate markers (one per gate), road approach markers on surrounding streets, amenity markers for water/restrooms/food/first aid.
4. Organiser assigns roads to gates: for each gate, selects which road markers lead to it.
5. Organiser draws the perimeter circle: clicks 'Set Perimeter', clicks centre point on map, drags to set radius. Radius is shown in metres. Saves.
6. Organiser places outer gathering zone markers: at least one per gate, minimum 500m from perimeter. Sets zone name and max capacity.
7. Organiser registers transport fleet: adds each buggy/bus with ID, type, capacity, and route.
8. Organiser clicks 'Generate Road Plan'. System calls Prompt 1 via Gemini API. Response rendered as map overlays + text report. Organiser reviews and saves.
9. Organiser clicks 'Generate Batch Schedule'. Enters total attendees and transport capacity. System calls Prompt 2 via Gemini API. Full schedule table rendered. Organiser reviews, adjusts if needed, saves.
10. Organiser clicks 'Publish Schedule'. System sets schedulePublished = true. All authenticated attendees can now see their batch assignments and begin slot booking.

## 4.2 Attendee Pre-Entry Flow

This flow is completed by the attendee before arriving at the venue.

11. Attendee navigates to VenueOS URL (or scans QR on physical ticket). App loads the auth screen.
12. Attendee enters their registered mobile number. Firebase Auth sends OTP SMS.
13. Attendee enters OTP. Firebase Auth verifies. System looks up attendee record in Firestore by phone number. If not found, prompts attendee to verify ticket ID to create profile.
14. Home screen loads. If schedule is not yet published, attendee sees a waiting screen with event date/time. If published, attendee sees slot selection screen.
15. Attendee selects entry slot from available windows. Attendee selects exit slot. Confirms booking.
16. System assigns batch code, gate, and gathering zone. Calls Cloud Function to generate signed QR pass payload. Writes all data to Firestore attendee document.

17. Home screen updates: shows QR pass, batch code, gathering zone name and address, entry window countdown, and a 'Navigate' button.
18. On day of event, attendee taps 'Navigate' to get directions to their gathering zone. App uses Google Maps Directions API for routing.

## 4.3 Event Day Entry Flow (Complete)

This is the full end-to-end flow on event day from gathering zone to seat.

19. Attendee arrives at outer gathering zone as per their batch code. The zone is a large open space 500m+ from the venue.
20. Organiser's dashboard shows all batches and their status. When a batch's entry window is 20 minutes away, a 'Dispatch Transport' button activates.
21. Organiser clicks 'Dispatch'. System marks batch as 'In Transit'. Push notification sent to all attendees in that batch: 'Your transport is ready at [Zone Name]. Board now.'
22. At the gathering zone, a staff member operates the QR scanner on their phone. Each attendee scans their QR pass. System validates batch code and marks attendee as 'Boarded'. Capacity count updates in real time on the organiser dashboard.
23. Transport departs when batch is fully boarded (or at scheduled departure time, whichever comes first).
24. Transport arrives at venue perimeter. As attendees exit transport, attendee app detects entry into geo-fence. App shows green 'Cleared to enter. Proceed to [Gate X].'
25. At perimeter gate: staff QR scanner validates (a) entry window active, (b) attendee boarded, (c) signature valid. Marks attendee as 'Perimeter Cleared'. Any invalid scan shows error reason.
26. Attendee walks to their assigned gate. At gate: staff QR scanner does final entry scan. Marks attendee as 'Gate Entered' and 'Inside'. Crowd count for that gate updates.
27. Inside venue, attendee uses live map to find amenities, see crowd density zones, and navigate to seat section.

## 4.4 Real-Time Density Analysis Flow (Runs Every 60 Seconds)

28. Cloud Scheduler triggers the density analysis Cloud Function every 60 seconds during the event window (2 hours before start to 1 hour after end).
29. Cloud Function queries Firestore: (a) gps_pings in last 2 minutes grouped by zone (using geohash proximity), (b) scan_events in last 10 minutes grouped by checkpointId, (c) current attendee checkpoint stage counts.
30. For each gate zone, computes: current headcount (GPS pings within 50m), rate of change vs previous snapshot, percentage of gate capacity used.
31. Assembles density snapshot JSON. Fetches current match state (can be manually entered by organiser or read from a match data field in the event document).
32. Calls Gemini API with Prompt 3. Parses the JSON response.

33. Writes new density_snapshot document to Firestore. Organiser dashboard listens via Firestore onSnapshot and updates heatmap automatically.
34. For any prediction with confidence > 0.7 and severity 'high' or 'critical': creates an alert document in Firestore with status 'pending'. Dashboard shows alert card with approve/edit/dismiss options.
35. When organiser approves an alert: Cloud Function sends in-app notification to all attendees within the target radius of the congested zone.

## 4.5 Incident Report Flow

36. Attendee taps 'Report Issue' from live map or amenity finder.
37. Selects report type: General Incident or Amenity Issue. If Amenity Issue, amenity is pre-populated from map tap.
38. Enters description. Optionally uploads photo or short video (max 30s). Media uploads to Cloud Storage. Upload URL stored temporarily.
39. Taps Submit. Backend API endpoint /attendee/:uid/report receives the report with all fields plus attendee's current GPS location.
40. Backend calls Gemini API with Prompt 4. Gemini returns category, priority, routedToRole, and actionRequired.
41. Backend writes complete incident_reports document to Firestore with Gemini classification fields populated.
42. Organiser dashboard shows new report card in the incidents panel, highlighted by priority colour. Staff of the routed role see the report in their staff view.
43. Attendee sees confirmation: 'Report #[ID] received. Priority: [level]. Estimated response: [time].'

## 4.6 Post-Match Exit Flow

44. 30 minutes before estimated match end, organiser activates 'Exit Mode' on dashboard.
45. System sends push notifications to all attendees: 'Your exit window is [time]. Please make your way to [Exit Gate X] when notified.'
46. Exit windows are staggered by batch — same logic as entry. Batches are notified when their exit window is active.
47. Attendee navigates to exit gate using live map. At exit gate, staff QR scanner validates exit window is active and marks attendee as 'Exited'.
48. If attendee tries to exit before their window, scanner shows amber warning: 'Exit window: [time]. Please wait in designated waiting area.'
49. Organiser can override any exit restriction for individual attendees from the dashboard (emergency override).
50. Transport returns from venue to outer gathering zones for attendees who came by the transport system.

# 5. UI Screen Specifications

Every screen in both applications with layout and content requirements

## 5.1 Organiser Dashboard Screens

### Screen O1 — Login

| | |
|---|---|
| **Layout** | Centred card on full-page background |
| **Components** | VenueOS logo, 'Sign in with Google' button (Firebase UI), app tagline |
| **Behaviour** | On successful Google sign-in, check Firestore admins collection. If authorised, redirect to O2. If not, show 'Access denied' with contact email. |

### Screen O2 — Event List / Home

| | |
|---|---|
| **Layout** | Top navbar with logo and logout. Main content: list of events as cards. |
| **Components** | Event cards showing: venue name, date, capacity, status badge (Planning \| Active \| Completed). 'Create New Event' button. |
| **Behaviour** | Click event card navigates to O3. Create New Event opens a modal form. |

### Screen O3 — Event Setup (Map & Gate Planner)

| | |
|---|---|
| **Layout** | Left sidebar (360px) for controls. Right panel: full-height Google Map. |
| **Sidebar tabs** | Setup \| Gates \| Perimeter \| Zones \| Transport |
| **Setup tab** | Event name, date, start time, total capacity fields. Save button. |
| **Gates tab** | List of gates with Add Gate button. Each gate: name, capacity, 'Place on Map' button (drops marker), road assignment multi-select. |
| **Perimeter tab** | 'Draw Perimeter' button — click map to set centre, drag to set radius. Shows radius in metres. Save button. |
| **Zones tab** | List of gathering zones with Add Zone button. Each zone: name, max capacity, 'Place on Map' button. |
| **Transport tab** | List of transport units. Add Transport button: ID, type (buggy/bus/shuttle), capacity, assigned route dropdown. |
| **Map behaviour** | Google Map with drawing tools enabled. Gate markers show gate name label. Perimeter shown as a dashed circle. Zone markers shown as a distinct icon. Amenity markers shown by type icon. |

| | |
|---|---|
| **AI Plan button** | Fixed at bottom of sidebar. 'Generate Road Plan' — calls API, shows loading spinner, then renders plan text in a modal + colour overlays on map. |

## Screen O4 — Batch Scheduler

| | |
|---|---|
| **Layout** | Full-width page with header controls and a schedule table below. |
| **Header** | Input fields: Total Attendees, Transport Capacity per Trip. 'Generate Schedule' button. |
| **Schedule table** | Columns: Batch Code \| Gate \| Gathering Zone \| Entry Window \| Exit Window \| Transport Departure \| Status. Each row is editable inline. |
| **Status badges** | Pending (grey) \| Dispatched (blue) \| In Transit (amber) \| Entered (green) |
| **Publish button** | Top right. Disabled until schedule has at least one batch. Confirm dialog before publishing. |
| **Dispatch column** | On event day, each row shows a 'Dispatch' button when the entry window is within 20 minutes. |

## Screen O5 — Live Dashboard (Event Day)

| | |
|---|---|
| **Layout** | Three-column layout. Left: alerts and staff instructions. Centre: Google Map with heatmap. Right: gate counters and batch status. |
| **Left panel** | Alert cards stacked vertically. Each card: zone name, severity badge, Gemini recommendation, Approve / Edit / Dismiss buttons. Approved alerts show 'Sent' status. |
| **Centre map** | Google Maps with heatmap layer. Gate markers with live count badges. Clicking a gate marker shows a popup with detailed stats. Gathering zone markers. Perimeter boundary. |
| **Right panel** | Per-gate counter cards: gate name, current queue estimate, entered in last 10 min, capacity %. Colour coded green/amber/red. Below: batch status list — shows each batch and their current stage. |
| **Incident panel** | Bottom drawer that slides up. Shows incident report cards sorted by priority. Red = urgent, amber = high, grey = medium/low. Click to expand details. |
| **Match state input** | Top bar: text field for organiser to update match state (e.g. '15th over', 'drinks break'). Used in Gemini prompt. |

## Screen O6 — QR Scanner (Staff)

| | |
|---|---|
| **Access** | Direct URL /scanner?token=[staff_token]. No login required. |
| **Layout** | Full-screen camera view with scan overlay frame. Result panel below. |
| **Checkpoint selector** | Dropdown at top: Transport Boarding \| Perimeter Gate \| Entry Gate \| Exit Gate. Staff selects which checkpoint they are operating. |

| | |
|---|---|
| **Scan result** | Green full-screen flash + attendee name + checkpoint stage for success. Red flash + error message for failure. Auto-resets after 2 seconds for next scan. |
| **Offline resilience** | Scanner caches last 100 validated QR payloads locally. If network drops, can validate HMAC signature offline. Queues scan events and syncs when reconnected. |

## 5.2 Attendee Mobile Web App Screens

### Screen A1 — Authentication

| | |
|---|---|
| **Layout** | Full-screen centred card. Mobile-optimised (max-width 420px). |
| **Components** | VenueOS logo + event name/date, mobile number input (E.164 with country code selector), 'Send OTP' button, OTP input (6-digit, auto-advances), 'Verify' button. |
| **Behaviour** | Auto-focus OTP field on display. Show resend timer (60 seconds). On success, navigate to A2 or A3. |

### Screen A2 — Slot Booking

| | |
|---|---|
| **Layout** | Scrollable card layout. Mobile-optimised. |
| **Components** | Event banner (name, date, venue). Section: 'Choose Your Entry Window' — list of available time slots as selectable cards showing time range and capacity remaining. Section: 'Choose Your Exit Window' — same format. 'Confirm Booking' button (disabled until both selected). |
| **Behaviour** | Slots at capacity are shown as greyed-out with 'Full' badge. Selection highlighted in blue. On confirm, spinner while API call processes, then navigate to A3. |

### Screen A3 — Home / Pass Screen

| | |
|---|---|
| **Layout** | Sticky top bar with event name. Scrollable content below. Bottom navigation bar. |
| **QR pass card** | Large card at top. Shows QR code (300x300px minimum). Below QR: Batch Code badge, Gate badge, Gathering Zone name. Tap QR to expand full-screen. |
| **Status bar** | Horizontal progress indicator: Booked > Boarded > Perimeter > Gate Entry > Inside > Exited. Current stage highlighted. |
| **Info cards** | Entry window countdown (or 'Window Active' if current). Exit window time. Gathering Zone address with 'Navigate' button. |

| | |
|---|---|
| **Notifications** | Recent notifications listed below info cards. |

## Screen A4 — Navigation

| | |
|---|---|
| **Layout** | Full-screen Google Map with bottom sheet for instructions. |
| **Destination logic** | Before perimeter entry: navigate to gathering zone. Inside perimeter but not gate entered: navigate to assigned gate. Inside venue: navigate to seat section (if seat data available). |
| **Map controls** | Recenter button, zoom controls, toggle between map and satellite view. |
| **Bottom sheet** | Walking/driving toggle. Estimated arrival time. Step-by-step directions list. |

## Screen A5 — Live Venue Map

| | |
|---|---|
| **Layout** | Full-screen Google Map with floating controls and bottom sheet. |
| **Map layers** | Gate markers (with name labels). Amenity markers (colour-coded icons: blue=water, green=first aid, orange=food, grey=restroom). Gathering zone markers. Perimeter boundary dashed circle. Crowd density colour overlay (semi-transparent zones). User location blue dot. |
| **Legend** | Floating bottom-left legend card for density colours and amenity icons. |
| **Tap behaviour** | Tap gate marker: shows gate name, queue estimate. Tap amenity marker: shows amenity name, status badge (open/issue/closed), 'Navigate Here' and 'Report Issue' buttons. |
| **Filter chips** | Horizontal scroll row at top: All \| Water \| Restrooms \| Food \| First Aid \| Exits. Filters visible amenity markers. |

## Screen A6 — Incident Report Form

| | |
|---|---|
| **Layout** | Full-screen form, mobile-optimised, scrollable. |
| **Report type toggle** | Two-button toggle: 'General Incident' \| 'Amenity Issue' |
| **General incident fields** | Description textarea (required). Photo/video upload button (optional, max 30s video). Submit button. |
| **Amenity issue fields** | Amenity selector (pre-filled if opened from map). Issue type dropdown: Broken / Out of stock / Unclean / Other. Description textarea. Photo upload. Submit. |
| **Post-submit** | Success screen with report reference number and estimated response time from Gemini. |

# 6. Deployment & Environment Setup

Cloud Run configuration, environment variables, and CI/CD

## 6.1 Environment Variables

Set all of the following as Cloud Run environment variables and as .env.local for local development. Never commit these to GitHub.

| Variable | Description |
|---|---|
| GEMINI_API_KEY | Google AI Studio API key for Gemini API access |
| FIREBASE_PROJECT_ID | Firebase/GCP project ID |
| FIREBASE_SERVICE_ACCOUNT_JSON | Base64-encoded Firebase Admin SDK service account JSON |
| HMAC_SECRET | Secret key for QR pass signing — minimum 32 characters, random |
| GOOGLE_MAPS_API_KEY | Maps Platform API key (restrict to your domain in GCP Console) |
| CLOUD_STORAGE_BUCKET | Cloud Storage bucket name for incident media uploads |
| ADMIN_EMAILS | Comma-separated list of Google account emails allowed organiser access |
| STAFF_TOKEN_SECRET | Secret for signing staff scanner JWT tokens |
| EVENT_ID | Default event ID for the current hackathon demo event |

## 6.2 Cloud Run Deployment

51. Build Docker image: docker build -t gcr.io/[PROJECT_ID]/venueos-backend .
52. Push to Container Registry: docker push gcr.io/[PROJECT_ID]/venueos-backend
53. Deploy to Cloud Run: gcloud run deploy venueos-backend --image gcr.io/[PROJECT_ID]/venueos-backend --platform managed --region asia-south1 --allow-unauthenticated --set-env-vars [all env vars]
54. Set minimum instances to 1 to prevent cold starts on event day: --min-instances 1
55. Set memory to 512Mi and CPU to 1: --memory 512Mi --cpu 1
56. Cloud Run URL is the base URL for all API calls. Update the frontend config with this URL.

## 6.3 Firestore Setup

57. Create Firestore database in Native mode in the Firebase console.
58. Enable the following Firestore indexes (composite): gps_pings: [eventId ASC, timestamp DESC], scan_events: [eventId ASC, checkpointType ASC, scannedAt DESC], density_snapshots: [eventId ASC, timestamp DESC], incident_reports: [eventId ASC, geminiPriority ASC, createdAt DESC].
59. Set Firestore TTL policy on gps_pings collection: field 'expireAt' as the TTL field.
60. Deploy Firestore security rules from firestore.rules in the repository.

## 6.4 GitHub CI/CD (cloudbuild.yaml)

| | |
|---|---|
| **cloudbuild.yaml** | steps: - name: 'gcr.io/cloud-builders/docker'  args: ['build', '-t', 'gcr.io/$PROJECT_ID/venueos-backend', './backend'] - name: 'gcr.io/cloud-builders/docker'  args: ['push', 'gcr.io/$PROJECT_ID/venueos-backend'] - name: 'gcr.io/google.com/cloudsdktool/cloud-sdk'  args: ['gcloud', 'run', 'deploy', 'venueos-backend', '--image', 'gcr.io/$PROJECT_ID/venueos-backend', '--region', 'asia-south1', '--platform', 'managed'] images: ['gcr.io/$PROJECT_ID/venueos-backend'] |